

AI Native Engineering Platform Blueprint

RAG / Agent / Eval / Observability 长格式测试文档 - 约 10 页

1. 文档目的和测试范围

本文档是一份面向 RAG 与 Agent 工程测试的长格式样例资料。它模拟一家企业准备建设 AI Native Engineering Platform 时产生的需求说明、系统架构、权限策略、评估体系和上线计划。文档内容故意包含中文段落、英文术语、编号列表、表格、风险项、阶段目标和跨页章节，适合用来测试 PDF 解析、chunk 切分、embedding 检索、metadata 保留、source citation 与问答 groundedness。

本样例不是某家真实公司的内部资料，但写法接近企业交付文档。建议在 ingestion 后检查 source、page、chunk_id、section_title、doc_type 等 metadata 是否保留，并对正文型问题、表格型问题、跨章节问题分别建立 retrieval eval。

- 测试目标一：验证多页中文 PDF 能否稳定解析，不出现大面积乱码。
- 测试目标二：观察标题、页脚、表格、编号列表是否进入 chunk，并评估是否需要清洗。
- 测试目标三：构造跨文档问题，例如把本需求说明与 Excel 中的供应商评分做联合查询。

2. 业务背景：研发团队为什么需要 Agent 平台

大型研发组织的痛点通常不是“缺少一个聊天机器人”，而是知识、代码、需求、测试、部署和审批分散在不同系统中。工程师需要在 Jira、飞书、GitHub、Confluence、日志平台和 CI 系统之间频繁切换，信息断裂会导致交付周期拉长、事故复盘困难、知识沉淀不足。

AI Native Engineering Platform 的目标是把 Agent 接入研发工作流，让它能够读取需求文档、理解代码仓库、执行受控工具、生成测试建议、追踪风险，并在关键节点产出可审计的证据链。平台不是替代工程师，而是把重复性检索、整理、校验和报告生成工作自动化。

业务对象	当前问题	Agent 可介入环节	成功指标
产品经理	需求背景散落在多个文档中	需求摘要、冲突检测、验收标准生成	需求澄清时间下降 30%
后端工程师	修改影响范围难以判断	代码搜索、调用链分析、测试建议	PR review 周期下降 20%
QA 团队	测试用例覆盖依赖经验	用例补全、边界条件生成、失败回放	缺陷逃逸率下降 15%
运维/SRE	告警上下文不足	日志摘要、变更关联、runbook 推荐	MTTR 下降 25%

3. 用户角色与权限边界

企业级 Agent 系统必须先定义权限，而不是先追求“智能”。任何能够访问代码、文档、工单或生产日志的 Agent，都应遵守最小权限原则。用户身份、组织角色、项目空间、数据等级和工具调用范围需要在运行时共同决定。

本平台将用户分为 Viewer、Contributor、Maintainer、Admin 四类角色。Viewer 只能发起问答与读取公开空间；Contributor 可以在项目空间内创建任务和草稿；Maintainer 可以批准 Agent 运行受控工具；Admin 管理连接器、审计规则和数据保留策略。

角色	可读数据	可执行操作	工具权限	审计要求
Viewer	公开文档、已授权知识库	问答、摘要	无写操作工具	记录 query 与 source
Contributor	项目文档、issue、PR	生成草稿、创建任务	只读代码搜索、文档检索	记录 prompt、tool result
Maintainer	项目代码、CI 日志	批准修改建议、触发测试	受控 terminal、CI rerun	记录审批人和 diff
Admin	组织配置、审计日志	管理连接器和策略	配置级工具权限	保留 180 天以上

3.1 权限校验原则

每次 tool call 之前都应进行授权检查。授权检查不能只发生在 UI 层，也不能只依赖 prompt 约束。系统应在后端 tool gateway 中校验 user_id、workspace_id、resource_id、action_type 和 risk_level。高风险工具必须要求显式审批，例如删除文件、执行数据库写操作、调用生产环境脚本。

4. 系统架构概览

平台采用分层架构：前端工作台负责交互和可视化，后端 orchestrator 管理 Agent 状态，tool gateway 执行受控工具调用，retrieval service 提供 RAG 检索，eval service 负责离线回放与质量评估。所有关键链路都需要记录 trace_id，以便出现错误时进行复盘。

对于工程面试或 FDE demo，架构图可以被简化为五个模块：User Workspace、Agent Orchestrator、Tool Gateway、Knowledge Index、Eval & Observability。重点不是组件数量，而是说明每个模块如何降低 Agent 幻觉、越权、重复调用和成本失控。

模块	核心职责	关键数据	失败模式
Agent Orchestrator	管理 plan、state、step、termination	run_id、step_id、state_json	循环失控、状态污染
Tool Gateway	统一工具注册、授权、超时、重试	tool_name、args、result	越权调用、脏数据返回
Knowledge Index	文档解析、chunk、embedding、检索	chunk_id、source、page	召回失败、证据噪声
Eval Service	离线 golden cases 和回放	expected_source、actual_source	只看最终答案不看过程
Observability	成本、延迟、错误、用户反馈	latency_ms、tokens、cost	问题不可定位

5. RAG Ingestion 设计要求

RAG ingestion

不是简单地把文件丢进向量库。生产级流程应包括文件识别、解析、清洗、结构化切分、表格处理、metadata 生成、embedding、索引写入和质量抽检。对于

PDF，应抽样检查中文编码、页眉页脚、目录残留、表格摊平、标题正文混杂和句子硬切断。

建议对不同文件类型采用不同 parser。普通 PDF 可使用 PyMuPDF 或 pdfplumber；复杂表格文档应使用 table-aware parser；Excel 应按 sheet、row range 和 semantic block 建立 chunk；Markdown 和代码仓库可以按照标题、函数、类和文件路径切分。

文件类型	解析策略	推荐 metadata	质量检查
PDF 正文	按页提取 + 句子感知切分	source/page/chunk_id/section	乱码、页脚、断句
PDF 表格	表格转 markdown 或 JSON	table_id/page/row_count	列关系是否保留
Excel	按 sheet 和业务区域切分	sheet_name/range/header	公式、空行、合并单元格
代码	按文件/函数/类切分	repo/path/language/symbol	上下文是否足够

5.1 Chunk 质量门禁

低质量 chunk 会直接影响 embedding 和 retrieval。系统应过滤过短文本、纯页码、目录点线、重复页眉页脚和没有语义的碎片。对于重要文档，应保留原始文本、清洗后文本和 parser 版本，避免后续无法复现检索结果。

- 最低长度阈值：中文正文建议不少于 80-120 字。
- 目录检测：包含大量点线和页码的 chunk 应标记为 toc_noise。
- 表格检测：包含多列枚举、金额、评分、风险矩阵时标记为 table_like。
- 可追溯性：每个答案必须能够追溯到 source、page 和 chunk_id。

6. Agent Workflow 与 Tool Calling

Agent 工作流应围绕任务状态而不是单次 prompt 组织。每次运行至少包含 objective、plan、steps、observations、artifacts、final_answer 和 failure_reason。复杂任务可以采用 Plan-Execute-Replan 模式：先规划，再执行工具调用，观察结果后决定是否继续、修正或终止。

工具 schema 设计要足够窄，参数明确，返回结果可压缩。搜索工具不应返回整篇网页，文件工具不应默认返回整份文档，terminal 工具必须有工作目录、命令白名单和超时限制。

工具	输入参数	输出结构	风险控制
search_docs	query, top_k, filters	chunks[], score, metadata	限制 top_k，记录 score
read_file	path, start_line, end_line	content, checksum	路径白名单
run_tests	repo, test_target	exit_code, stdout, stderr	超时和资源限制
create_ticket	project, title, body	ticket_id, url	需要用户确认

7. Evaluation Harness

Agent 评估不能只看最终回答是否流畅，而要评估 trajectory。一个高质量 eval harness 应检查检索是否命中正确证据、工具调用是否合理、是否出现重复无效调用、是否遵守权限、是否在证据不足时给出 fallback，以及最终回答是否包含可验证引用。

建议维护三类 golden cases：事实检索型、流程执行型和失败兜底型。事实检索型关注 answer-source matching；流程执行型关注工具顺序和状态转换；失败兜底型关注系统是否能承认缺少证据，而不是强行编造。

Eval 类型	检查对象	通过标准	失败例子
Retrieval Eval	top-k chunks	top-3 包含正确 source	命中目录而非正文
Trajectory Eval	tool calls	步骤合理且无重复循环	同一 query 搜索 6 次
Answer Eval	final answer	答案有引用且不越界	引用不存在的页码
Safety Eval	permissions	高风险工具需要审批	自动删除用户文件

8. 可观测性与成本控制

Agent 平台上线后，最常见的问题不是模型完全不可用，而是慢、贵、偶发错误和结果不可解释。因此每次 run 都应记录 token、cost、latency、retrieval time、tool time、model name、retry count、fallback reason。没有这些数据，团队无法判断是模型问题、检索问题还是工具问题。

成本控制可以从三个方向做：减少无效上下文、减少重复工具调用、为不同任务选择不同模型。简单分类、摘要、格式化任务不一定需要最强模型；高风险决策、复杂规划和最终交付报告可以使用更强模型。

指标	含义	报警阈值示例	排查方向
latency_ms	端到端延迟	> 30s	工具慢或模型慢
retrieval_hit_rate	检索命中率	< 80%	chunk 或 query rewrite 问题
tool_error_rate	工具失败率	> 5%	权限、超时、接口异常
cost_per_run	单次运行成本	> 预期 2 倍	上下文过长或循环

9. 上线计划与验收标准

上线应分为内部 dogfood、试点团队、灰度扩展和组织级推广四个阶段。每个阶段都要设置明确的功能边界和回滚机制。第一阶段只允许只读检索和报告生成；第二阶段允许创建 issue 或草稿；第三阶段才逐步开放受控代码修改和 CI 工具。

验收标准不应只写“支持智能问答”。更合理的标准是：指定 50 条 golden queries，top-3 retrieval accuracy 不低于 85%；关键工具调用失败率低于 3%；所有最终答案必须包含 source；高风险工具调用必须有人工审批记录。

阶段	范围	开放能力	验收指标
Phase 0	平台团队	文档问答、日志摘要	核心查询命中率 80%
Phase 1	两个试点项目	需求分析、测试建议	用户满意度 4/5
Phase 2	研发部门	PR 摘要、CI 辅助	节省工时可量化
Phase 3	组织级	跨系统工作流	审计和权限全覆盖

10. 附录：建议检索问题

以下问题用于测试该文档的 RAG 检索效果。建议先不接 LLM，只看 top-k chunk 是否命中正确章节。然后再接 answer generation，检查答案是否能正确引用 source/page/chunk_id。

Query	Expected Section	Expected Evidence
企业级 Agent 平台为什么不能只做聊天机器人？	第 2 章	研发系统分散、信息断裂、工作流自动化
Maintainer 能执行哪些操作？	第 3 章	批准修改建议、触发测试、受控工具
RAG ingestion 的质量门禁包括哪些？	第 5 章	乱码、页脚、断句、表格、metadata
Agent eval 为什么要看 trajectory？	第 7 章	工具调用、重复循环、权限、fallback
上线第一阶段应该开放什么能力？	第 9 章	只读检索和报告生成